



第十四章:交易

数据库系统概念, 6thEd。

©Silberschatz, Korth和Sudarshan参见www.db-book.com了解
重用条件



第十四章:交易

“交易概念”

·交易状态

并发执行

☒可串行性

☒可恢复性

“隔离”在SQL中的“事务定义”
的实现“序列化性测试”。



事务的概念

事务是程序执行的一个单元，它访问并可能更新各种数据项。

例如，从账户A向账户B转账50美元的交易：

1. 阅读(一)

2. $A := A - 50$

3. 写(一个)

4. 读(B)

5. $B := B + 50$

6. 写(B)



需要处理的两个主要问题：

z 各种各样的故障，比如硬件故障和系统崩溃

z 多个事务的并发执行



资金转帐示例

从账户A向账户B转账50美元的交易:

1. 阅读(一)

2. $A := A - 50$

3. 写(一个)

4. 读(B)

5. $B := B + 50$

6. 写(B)

“原子性要求”

如果事务在步骤3之后和步骤6之前失败，资金将“丢失”，导致数据库状态不一致

失败可能是由于软件或硬件的原因

系统应确保部分执行的事务的更新不会反映在数据库中

持久性要求——一旦用户被通知交易已经完成(即50美元的转账已经发生)，即使出现软件或硬件故障，交易对数据库的更新也必须持续。



资金转帐示例(续)

“从账户A向账户B转账50美元的交易:

1. 阅读(一)
2. $A := A - 50$
3. 写(一个)
4. 读(B)
5. $B := B + 50$
6. 写(B)

上面例子中的一致性要求:

z A和B的总和因交易的执行而不变。一般而言，一致性要求包括

显式指定的完整性约束，如主键和外键

隐式完整性约束

如。所有账户余额的总和减去贷款金额的总和必须等于手头现金的价值

一笔交易必须有一个一致的数据库。

在事务执行期间，数据库可能暂时不一致。

当事务成功完成时，数据库必须保持一致

错误的事务逻辑会导致不一致



资金转移示例(续)

- ☒ **隔离要求**-如果在步骤3和6之间，另一个事务T2被允许访问部分更新的数据库，它将看到一个不一致的数据库($A + B$ 的总和将小于它应该是)。

T1 T2

1. 阅读(一)

2. $A := A - 50$

3. 写(一个)

读(A), 读(B), 打印($A+B$)

4. 读(B)

5. $B := B + 50$

6. 写(B)

通过连续运行事务(即一个接一个地**运行事务**), 可以轻松地确保隔离。

然而, 并发执行多个事务有显著的好处, 我们将在后面看到。



ACID属性

事务是访问并可能更新各种数据项的程序执行单元。为了保持数据的完整性，数据库系统必须保证：

- ☒ **原子性**。要么事务的所有操作都正确地反映在数据库中，要么没有。
- ☒ **一致性**。隔离地执行事务可以保持数据库的一致性。
- ☒ **隔离**。虽然多个事务可以并发执行，但每个事务必须不知道其他并发执行的事务。中间事务的结果必须对其他并发执行的事务隐藏。

z 也就是说，对于每一对事务 T_i 和 T_j ，在 T_i 看来，要么 T_j 在 T_i 开始之前完成执行，要么 T_j 在 T_i 结束之后开始执行。
- ☒ **耐久性**。事务成功完成后，即使出现系统故障，它对数据库所做的更改也会持续存在。



事务状态

Active -初始状态;事务在执行时保持在此状态

部分提交——在最后一条语句被执行之后。

失败——在发现正常的执行不能再继续进行之后。

流产——在事务回滚并且数据库恢复到事务开始之前的状态之后。
在事务被中止后有两个选项:

z重启事务

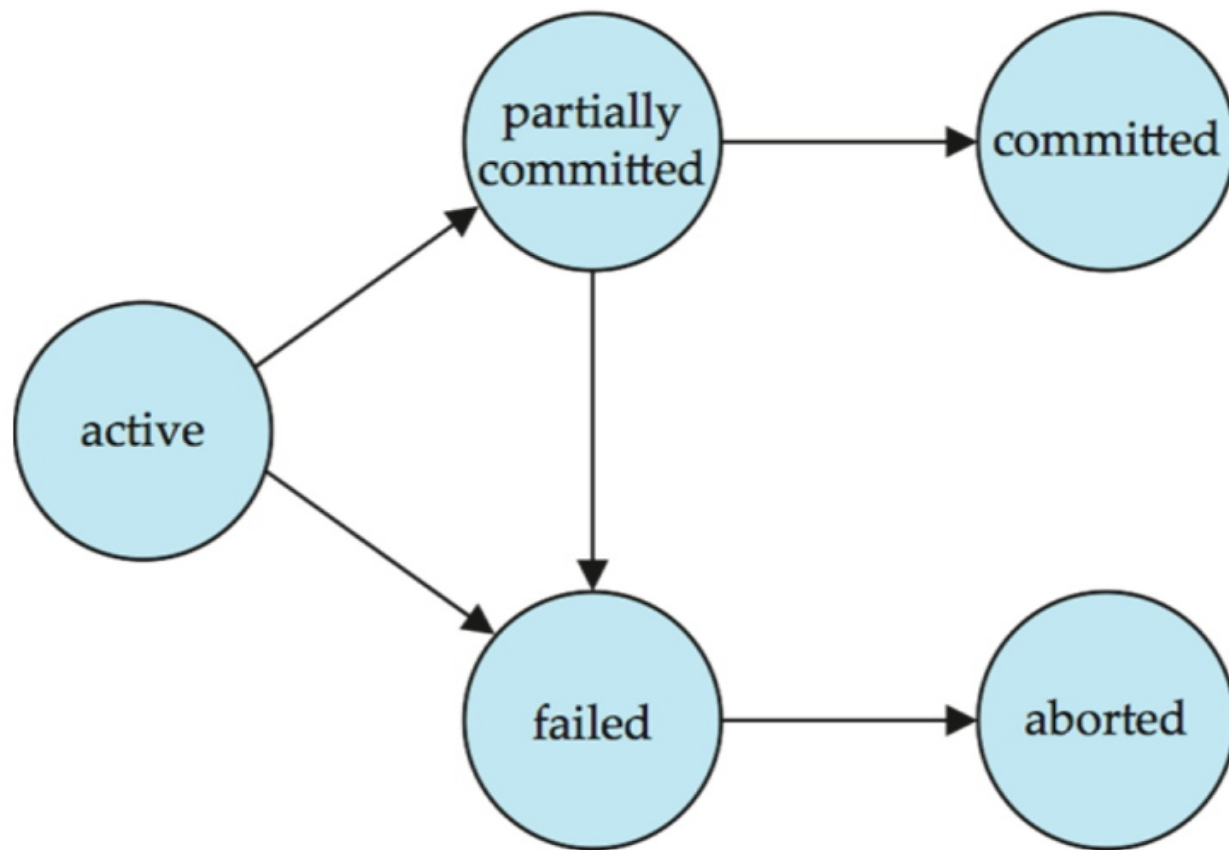
只能在没有内部逻辑错误的情况下进行

终止事务

*** Committed** -在成功完成后。



事务状态(续)





同时执行

允许多个事务在系统中并发运行。优点是:

- z 提高了处理器和磁盘利用率，导致了更好的事务吞吐量

例如，一个事务可以使用CPU，而另一个事务可以从磁盘读取或写入

- z 减少事务的平均响应时间:短事务不需要在长事务后面等待。

并发控制方案——实现隔离的机制

- z 即控制并发事务之间的交互，以防止它们破坏数据库的一致性

将在第16章学习，在学习了并发执行正确性的概念之后。



日程安排

☒ **时间表**-指令序列，指定并发事务的指令按时间顺序执行

—组事务的调度必须包含这些事务的所有指令

z必须保持指令在每个单独交易中出现的顺序。

成功完成其执行的事务将以提交指令作为最后一条语句

默认情况下，事务**z**假定执行提交指令作为其最后一步

未能成功完成其执行的事务将以中止指令作为最后一条语句



表1

设 T_1 将\$50从A转到B, T_2 将余额的10%从A转到B。

“ T_1 后面跟着 T_2 的连续调度:

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit



表2

- T_2 后接 T_1 的串行时间表

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit



Schedule 3

设 T_1 和 T_2 为前面定义的事务。下面的时间表不是一个串行时间表，但它**相当于**附表1。

T_1	T_2
read (A) $A := A - 50$ write (A)	read (A) $temp := A * 0.1$ $A := A - temp$ write (A)
read (B) $B := B + 50$ write (B) commit	read (B) $B := B + temp$ write (B) commit

在附表1、附表2和附表3中，保留了 $A + B$ 的总和。



Schedule 4

下面的并发调度不保留 $(A + B)$ 的值。

T_1	T_2
read (A) $A := A - 50$	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B)
write (A) read (B) $B := B + 50$ write (B) commit	$B := B + temp$ write (B) commit



可串行性

基本假设-每个事务保持数据库一致性。

因此，一组事务的串行执行保持了数据库的一致性。

如果一个(可能是并发的)调度相当于一个串行调度，那么它是可串行化的。不同形式的调度等价产生了以下概念：

1. **冲突可串行性**
2. **视图可串行性**



简化的事务视图

我们忽略读写指令以外的操作

我们假设在读写之间，事务可能会对本地缓冲区中的数据执行任意计算。

我们的简化调度只包括读和写指令。



相互冲突的指令

事务 T_i 和事务 T_j 的指令 l_i 和 l_j ，当且仅当存在某个项目 Q 同时被 l_i 和 l_j 访问，并且这些指令中至少有一条写 Q 时发生**冲突**。

1. $l_i = \text{read}(Q), l_j = \text{read}(Q)$ 。 l_i 和 l_j 不冲突。
2. $l_i = \text{读}(Q), l_j = \text{写}(Q)$ 。他们的冲突。
3. $l_i = \text{写}(Q), l_j = \text{读}(Q)$ 。他们的冲突
4. $l_i = \text{write}(Q), l_j = \text{write}(Q)$ 。他们的冲突

直观上， l_i 和 l_j 之间的冲突迫使它们之间形成(逻辑上的)时间顺序。

如果 l_i 和 l_j 在一个时间表中是连续的，并且它们不冲突，那么即使它们在时间表中互换了，它们的结果也会保持不变。



冲突可串行性

如果一个调度 S 可以通过一系列不冲突指令的交换转换成一个调度 S' ，我们说 S 和 S' 是**冲突等价的**。

如果一个调度 S 与一个**串行调度是冲突**等价的，我们说它是冲突可串行化的



冲突序列化性(Cont.)

通过一系列不冲突指令的交换，可将附表3转换为附表6，即 T_2 跟随 T_1 的串行时间表。因此，Schedule 3是可冲突序列化的。

T_1	T_2
read (A) write (A)	read (A) write (A)
read (B) write (B)	read (B) write (B)

Schedule 3

T_1	T_2
read (A) write (A) read (B) write (B)	read (A) write (A) read (B) write (B)

Schedule 6



Conflict Serializability (Cont.)

✘ 不可冲突序列化的时间表示例:

T_3	T_4
read (Q)	write (Q)
write (Q)	

✘ We are unable to swap instructions in the above schedule to
获取串行调度 $\langle T_3, T_4 \rangle$, 或串行调度 $\langle T_4, T_3 \rangle$ 。



V查看序列化性

- ☒ 设 S 和 S' 是两个具有相同交易集的时间表。如果满足以下三个条件， S 和 S' 是**视图等价**的，对于每个数据项 Q ，
1. 如果在调度 S 中，事务 T_i 读取了 Q 的初始值，那么在调度 S 中同样事务 T_i 也必须读取 Q 的初始值。
 2. 如果在调度 S 中事务 T_i 执行**读(Q)**，并且该值是由事务 T_j (如果有的话)产生的，那么在调度 S 中事务 T_i 也必须读取由事务 T_j 的相同**写(Q)**操作产生的 Q 值。
 3. 在调度 S 中执行最后**写(Q)**操作的事务(如果有的话)也必须在调度 S' 中执行最后**写(Q)**操作。

可以看到，视图等价也是单纯基于**读写**操作的。



V视图序列化性(续)

如果一个调度S相当于一个**串行调度**，那么它是可视图序列化的。

每个冲突可序列化的调度也是视图可序列化的。

下面是一个可视图序列化但不可冲突序列化的时间表。

T_{27}	T_{28}	T_{29}
read (Q)	write (Q)	
write (Q)		
		write (Q)

“上述系列计划相当于什么?”

每个非冲突串行化的视图串行化调度都有**盲写**。



可序列化性的其他概念

- ☒ 下面的时间表与串行时间表 $\langle T_1, T_5 \rangle$ 产生相同的结果，但不是冲突等效或视图等效。

T_1	T_5
read (A) $A := A - 50$ write (A)	
	read (B) $B := B - 10$ write (B)
read (B) $B := B + 50$ write (B)	
	read (A) $A := A + 10$ write (A)

确定这种等价性需要分析除读和写以外的操作。



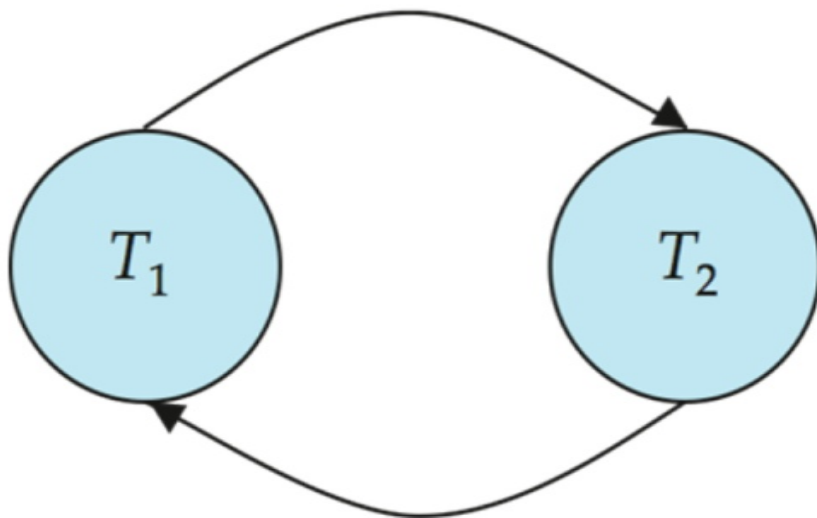
可序列化性测试

“考虑一组事务 $T_1, T_2, \dots, T_n$ ” **“优先图”**-一个直接图，其中顶点是事务(名称)。

如果两个事务冲突，我们绘制一条从 T_i 到 T_j 的弧，并且 T_i 访问了先前发生冲突的数据项。

我们可以用被访问的项目来标记弧。

“例1”





冲突序列化性测试

一个调度是可冲突序列化的当且仅当它的优先级图是无循环的。

存在花费 n 阶时间的循环检测算法，其中 n 是图中顶点的个数。

z(更好的算法需要 $n + e$ 阶，其中 e 是边的个数。)

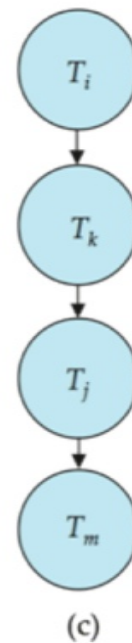
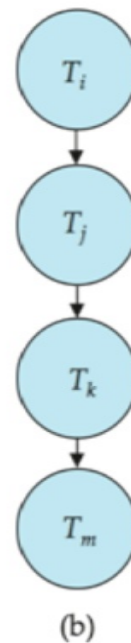
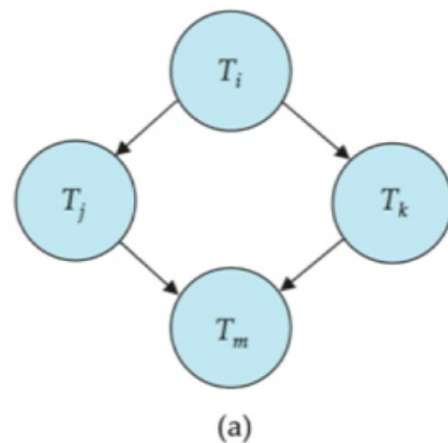
如果优先图是无环的，则可以通过对图进行拓扑排序来获得序列化顺序。

z这是与图的偏序一致的线性顺序。

z例如，调度a的可序列化性顺序为

$T_5 \rightarrow t_1 \rightarrow t_3 \rightarrow t_2 \rightarrow t_4$

还有其他的吗?





测试View Serializability

冲突序列化性的优先图测试不能直接用于测试视图序列化性。

用于测试视图序列化性的扩展在优先图的大小上花费指数。

*检查一个调度是否可视图序列化的问题属于 np 完全问题。

z因此，存在一个有效的算法是极不可能的。

然而，实用的算法，只是检查视图序列化性的一些充分条件，仍然可以使用。



可恢复时间表

需要解决事务失败对并发运行事务的影响。

可恢复调度-如果事务 T_j 读取先前由事务 T_i 写入的数据项，则 T_i 的提交操作出现在 T_j 的提交操作之前。

如果 T_9 提交，下面的调度(调度11)是不可恢复的
立即在read之后

T_8	T_9
read (A)	
write (A)	
	read (A)
	commit
read (B)	

如果 T_8 应该中止，那么 T_9 将读取(并可能显示给用户)不一致的数据库状态。因此，数据库必须确保调度是可恢复的。



Cascading Rollbacks

- ☒ **级联回滚**——单个事务失败导致一系列事务回滚。考虑下面的调度，其中还没有事务提交(因此调度是可恢复的)

T_{10}	T_{11}	T_{12}
read (A) read (B) write (A)	read (A) write (A)	read (A)
abort		

如果 T_{10} 失败, T_{11} 和 T_{12} 也必须回滚。会导致大量工作

- ☒ 的撤销吗



无级表

无级联调度——不能发生级联回滚;对于每一对事务 T_i 和 T_j , 使得 T_j 读取之前由 T_i 写入的数据项, T_i 的提交操作出现在 T_j 的读取操作之前。

每个无级联计划也是可恢复的

希望将调度限制为无级联的调度



并发控制

数据库必须提供一种机制，以确保所有可能的调度都是并行的

要么是冲突的，要么是视图序列化的

是可恢复的，最好是无级联的

“一次只能执行一个事务的策略生成串行调度，但提供的并发度很差
串行调度是可恢复的/无级联的吗？”

在调度执行后测试它的可序列化性有点晚了！

目标-开发并发控制协议，以确保可序列化性。



并发控制(续)

为了数据库一致性，调度必须是冲突或视图序列化的，并且是可恢复的，最好是无级联的。

一次只能执行一个事务的策略会生成串行调度，但提供的并发性能很差。

并发控制方案在它们允许的并发量和它们产生的开销之间进行权衡。

一些方案只允许生成可冲突序列化的调度，而另一些方案允许不可冲突序列化的可视图序列化的调度。



并行性控制与可序列化性测试

并发控制协议允许并发调度，但要确保调度是冲突/视图序列化的，并且是可恢复的和无级联的。

并发控制协议通常在创建优先级图时不检查优先级图

相反，协议强加了一个避免不可串行化调度的规则。

我们将在第16章学习这样的协议。

不同的并发控制协议在它们允许的并发量和它们产生的开销之间提供不同的权衡。

可序列化性的测试帮助我们理解为什么并发控制协议是正确的。



一致性的弱级别

一些应用程序愿意接受弱一致性级别，允许不可序列化的调度

例如，一个只读事务想要获得所有帐户的大致总余额

例如，用于查询优化的数据库统计数据可能是近似的(为什么?)

此类事务不需要相对于其他事务是可序列化的

为了性能而权衡准确性



SQL-92中的一致性级别

Serializable -default

可重复只读-只读提交的要读的记录，重复读同一条记录必须返回相同的值。然而，一个事务可能不是可序列化的——它可能会找到一些由事务插入的记录，但找不到其他记录。

读已提交-只能读已提交的记录，但是对记录的连续读可能返回不同的(但已提交的)值。

读未提交的记录——即使未提交的记录也可以被读。

较低的一致性度对于收集关于数据库的近似信息很有用

警告:默认情况下，一些数据库系统不确保可序列化的调度

例如，Oracle和PostgreSQL默认支持一种称为快照隔离的一致性级别(不是SQL标准的一部分)。



SQL中的事务定义

数据操作语言必须包含一个用于指定组成事务的一组操作的结构。

在SQL中，事务隐式地开始。

SQL中的事务以以下方式结束：

- 提交**工作提交当前事务并开始一个新的事务。

- z**回滚工作导致当前事务中止。

在几乎所有数据库系统中，默认情况下，如果成功执行，每个SQL语句也会隐式提交

- z**隐式提交可以通过数据库指令关闭，例如在JDBC中，
`connection.setAutoCommit(false);`



第十四章结束

数据库系统概念，6th编辑。

©Silberschatz, Korth和Sudarshan参见www.db-book.com了解
重用条件



Figure 14.01

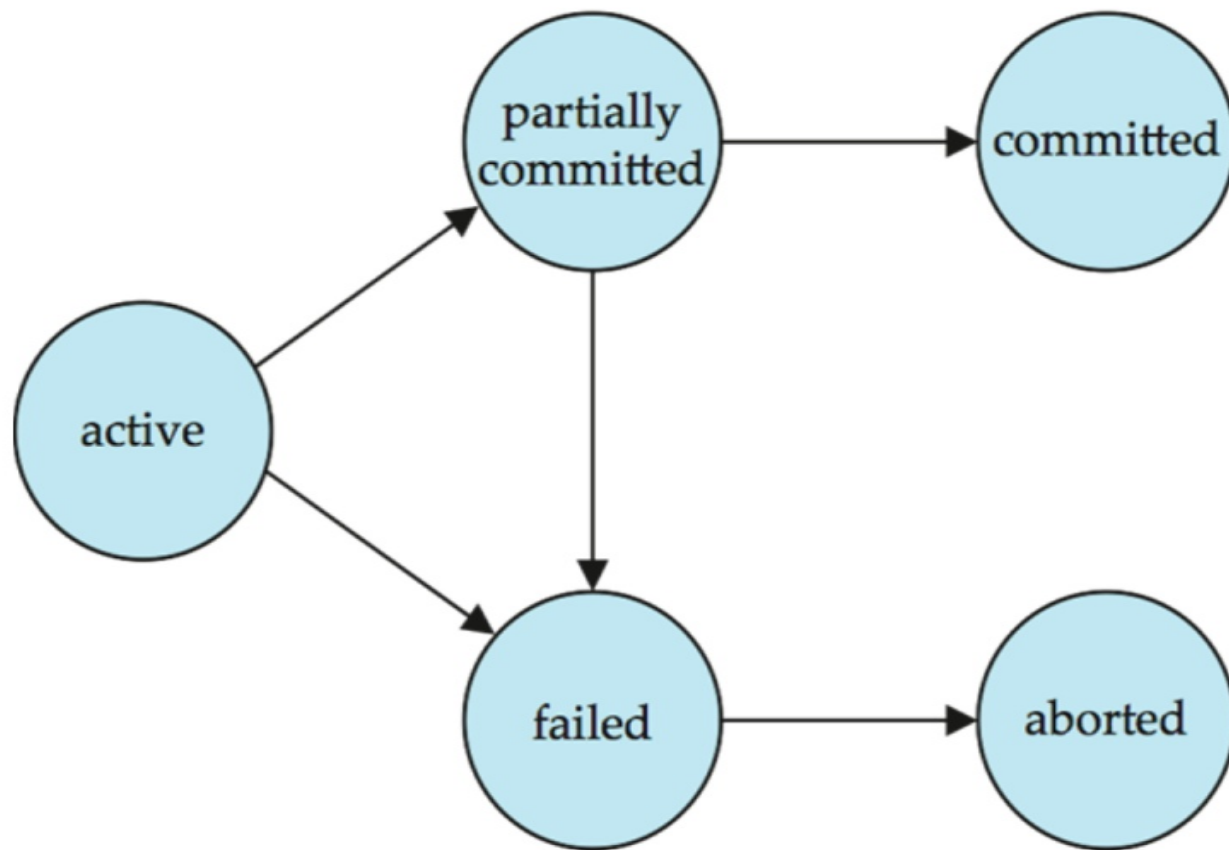




Figure 14.02

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit



```

read (A)
A := A - 50
write (A)
read (B)
B := B + 50
write (B)
commit

```




Figure 14.04

T_1	T_2
read (A) $A := A - 50$ write (A)	read (A) $temp := A * 0.1$ $A := A - temp$ write (A)
read (B) $B := B + 50$ write (B) commit	read (B) $B := B + temp$ write (B) commit

©Silberschatz, Korth and Sudarshan



Figure 14.06

T_1	T_2
read (A) write (A)	read (A) write (A)
read (B) write (B)	read (B) write (B)



Figure 14.07

T_1	T_2
read (A)	
write (A)	
	read (A)
read (B)	
	write (A)
write (B)	
	read (B)
	write (B)



Figure 14.08

T_1	T_2
read (A) write (A) read (B) write (B)	read (A) write (A) read (B) write (B)

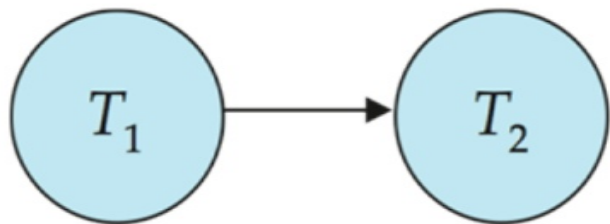


Figure 14.09

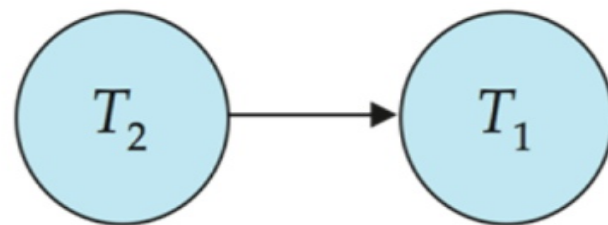
T_3	T_4
read (Q)	write (Q)
write (Q)	



图14.10



(a)



(b)



图14.11

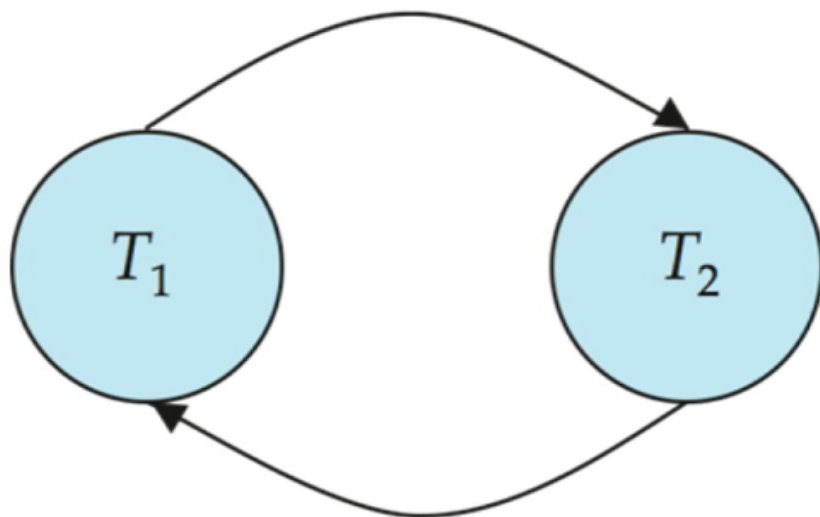
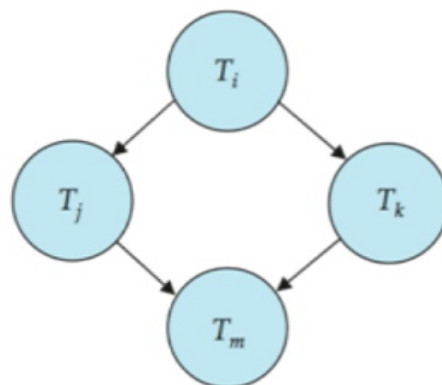
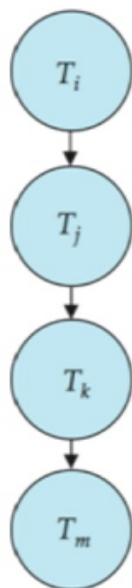




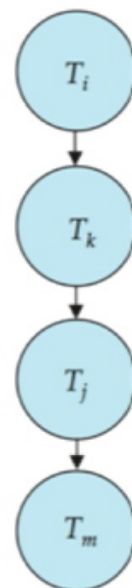
图14.12



(a)



(b)



(c)



Figure 14.13

T_1	T_5
read (A) $A := A - 50$ write (A)	
	read (B) $B := B - 10$ write (B)
read (B) $B := B + 50$ write (B)	
	read (A) $A := A + 10$ write (A)



Figure 14.14

T_8	T_9
read (A) write (A)	read (A) commit
read (B)	



Figure 14.15

T_{10}	T_{11}	T_{12}
read (A) read (B) write (A)	read (A) write (A)	read (A)
abort		



图14.16

